JALON 3

Modélisation de la Base de Données

Application de gestion et de suivi des animaux domestiques

PET CARE

Tristan DZIOCH
Formation CDA – Projet Fil Rouge
Mars 2026

Jalon	Échéance	Auteur	Version
3 – Modélisation BDD	31/03/2026	Tristan DZIOCH	1.0

1. Introduction à la méthode MERISE

Dans le cadre de ce projet, la modélisation de la base de données est réalisée selon la méthode MERISE. Cette approche consiste à concevoir la base en trois étapes complémentaires : le Modèle Conceptuel de Données (MCD), le Modèle Logique de Données (MLD) puis le Modèle Physique de Données (MPD). Cette progression du conceptuel vers le physique permet de garantir que la base répond aux besoins fonctionnels, tout en assurant la cohérence des relations, la normalisation des données et l'adaptation au SGBD retenu.

2. Dictionnaire des données

La base de données repose sur cinq entités principales : Utilisateur, Animal, Événement, Type_Événement et Partage_Animal.

Entité : Utilisateur

Attribut	Type	Contrainte	Description
id_utilisateur	INT	PK, AUTO_INCREMENT, NOT NULL	Identifiant unique de l'utilisateur
nom	VARCHAR(100)	NOT NULL	Nom de famille
prenom	VARCHAR(100)	NOT NULL	Prénom
email	VARCHAR(255)	UNIQUE, NOT NULL	Adresse email (identifiant de connexion)
mot_de_passe_hash	VARCHAR(255)	NOT NULL	Mot de passe haché (bcrypt/Argon2)
date_inscription	DATETIME	NOT NULL, DEFAULT NOW()	Date et heure d'inscription
role	VARCHAR(50)	NOT NULL, DEFAULT 'user'	Rôle de l'utilisateur dans l'application
photo_url	VARCHAR(500)	NULL	URL de la photo de profil (optionnel)

Entité : Animal

Attribut	Type	Contrainte	Description
id_animal	INT	PK, AUTO_INCREMENT, NOT NULL	Identifiant unique de l'animal
nom	VARCHAR(100)	NOT NULL	Nom de l'animal
especе	VARCHAR(100)	NOT NULL	Espèce (chien, chat, NAC…)
race	VARCHAR(100)	NULL	Race de l'animal (optionnel)
date_naissance	DATE	NULL	Date de naissance (optionnel)
date_inscription	DATE	NULL	Date d'inscription de l'animal (optionnel)
sexe	VARCHAR(10)	NULL	Sexe de l'animal (optionnel)
photo_url	VARCHAR(500)	NULL	URL de la photo de l'animal (optionnel)
id_utilisateur	INT	FK → Utilisateur, NOT NULLFK → Utilisateur, NOT NULL	Référence vers le propriétaire (utilisateur)

Entité : Événement

Attribut	Type	Contrainte	Description
id_evenement	INT	PK, AUTO_INCREMENT, NOT NULL	Identifiant unique de l'événement
id_animal	INT	FK → Animal, NOT NULL	Animal concerné par l'événement
id_type_evenement	INT	FK → Type_Evenement, NOT NULL	Catégorie de l'événement
id_createur	INT	FK → Utilisateur, NOT NULL	Utilisateur ayant créé l'événement
date_heure_evenement	DATETIME	NOT NULL	Date et heure de l'événement
statut	VARCHAR(50)	NOT NULL, DEFAULT 'planifié'	Statut de l'événement
rappel_actif	TINYINT(1)	NOT NULL, DEFAULT 0	Rappel activé (1) ou non (0)
rappel_jours_avant	INT	NULL	Nombre de jours avant envoi du rappel
commentaire	TEXT	NULL	Commentaire libre (optionnel)
date_creation	DATETIME	NOT NULL, DEFAULT NOW()	Date de création de l'enregistrement
date_modification	DATETIME	NULL	Date de dernière modification

Entité : Type_Événement

Attribut	Type	Contrainte	Description
id_type_evenement	INT	PK, AUTO_INCREMENT, NOT NULL	Identifiant unique du type
libelle	VARCHAR(100)	NOT NULL	Libellé du type (vaccin, RDV, traitement…)
description	TEXT	NULL	Description détaillée (optionnel)
couleur	VARCHAR(7)	NULL	Code couleur hexadécimal pour l'affichage (optionnel)

Entité : Partage_Animal

Attribut	Type	Contrainte	Description
id_animal	INT	PK, FK → Animal, NOT NULL	Référence vers l'animal partagé
id_utilisateur	INT	PK, FK → Utilisateur, NOT NULL	Référence vers l'utilisateur invité
role_partage	VARCHAR(50)	NOT NULL	Niveau d'accès attribué (lecture / modification)
date_invitation	DATETIME	NOT NULL, DEFAULT NOW()	Date à laquelle le partage a été créé

3. Modèle Conceptuel de Données (MCD)

3.1 Règles de gestion métier

Les règles de gestion suivantes ont guidé la conception du MCD. Elles définissent les contraintes métier fondamentales du domaine et conditionnent les cardinalités de chaque association.

1. Un utilisateur peut posséder de 0 à N animaux. Un animal appartient à exactement un propriétaire.
2. Un animal peut avoir de 0 à N événements. Un événement est rattaché à exactement un animal.
3. Un événement est catégorisé par exactement un type d'événement. Un type d'événement peut être associé à 0 à N événements.
4. Chaque événement est créé par exactement un utilisateur (traçabilité). Un utilisateur peut créer 0 à N événements.
5. Un utilisateur peut être invité à accéder à 0 à N animaux. Un animal peut être partagé avec 0 à N utilisateurs invités.
6. Un couple (id_animal, id_utilisateur) dans Partage_Animal est unique : un même utilisateur ne peut pas être invité deux fois sur le même animal.
7. Le propriétaire d'un animal ne peut pas figurer dans la table Partage_Animal pour ce même animal : il accède à ses propres animaux via la relation de propriété.
8. La suppression d'un animal entraîne la suppression en cascade de tous ses événements et de toutes ses entrées de partage (intégrité référentielle).

3.2 Diagramme MCD

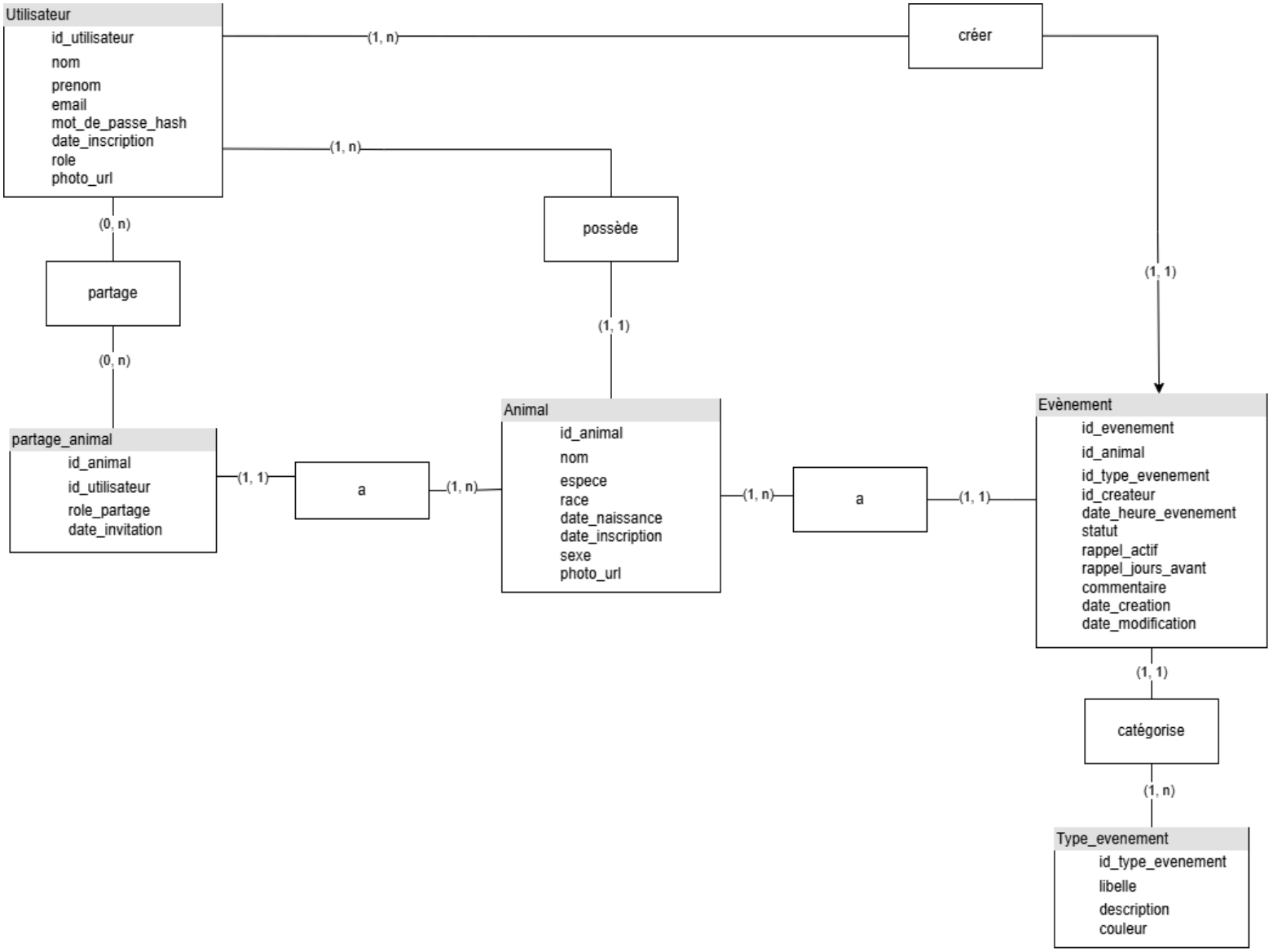
Le diagramme MCD ci-dessous représente les cinq entités du domaine métier et leurs associations. Le diagramme est disponible en haute résolution dans le dépôt GitHub du projet (dossier docs/) pour garantir une lecture optimale en projection ou format A4.

Lien GitHub (dossier docs/) : https://github.com/Nastry69/PetCare/tree/main/Docs

Les associations et leurs cardinalités sont les suivantes :

- **possède (Utilisateur → Animal) : (1,n) – (1,1) — un utilisateur possède un ou plusieurs animaux ; un animal appartient à exactement un propriétaire.**(Utilisateur → Animal) : 1,N – 1,1 — un utilisateur possède un ou plusieurs animaux ; un animal appartient à exactement un propriétaire.
- **a (Animal → Événement) : (1,n) – (1,1) — un animal peut avoir plusieurs événements ; un événement est lié à un seul animal.**(Animal → Événement) : 1,N – 1,1 — un animal peut avoir plusieurs événements ; un événement est lié à un seul animal.
- **catégorise (Événement → Type_Événement) : (1,1) – (1,n) — un événement est catégorisé par un seul type ; un type peut catégoriser plusieurs événements.**(Événement → Type_Événement) : 1,N – 1,1 — un événement est catégorisé par un seul type ; un type peut catégoriser plusieurs événements.
- **créer (Utilisateur → Événement) : (1,n) – (1,1) — un utilisateur peut créer plusieurs événements ; chaque événement a un créateur unique.**(Utilisateur → Événement) : 1,N – 1,1 — un utilisateur peut créer plusieurs événements ; chaque événement a un créateur unique.
- **partage (Utilisateur ↔ Animal via Partage_Animal) : (0,n) – (0,n) — association plusieurs-à-plusieurs avec attributs (role_partage, date_invitation).**(Utilisateur ↔ Animal via Partage_Animal) : 0,N – 0,N — association plusieurs-à-plusieurs avec attributs (role_partage, date_invitation).

MCD


Utilisateur
id_utilisateur
nom
prenom
email
mot_de_passe_hash
date_inscription
role
photo_url
(1, n)
créer
(1, n)
possède
(0, n)
partage
(1, 1)
(0, n)
(1, 1)
Evènement
id_evenement
id_animal
id_type_evenement
id_createur
date_heure_evenement
statut
rappel_actif
rappel_jours_avant
commentaire
date_creation
date_modification
Animal
id_animal
nom
espece
race
date_naissance
date_inscription
sexe
photo_url
partage_animal
id_animal
id_utilisateur
role_partage
date_invitation
(1, 1)
a
(1, n)
(1, n)
a
(1, 1)
(1, 1)
catégorise
(1, n)
Type_evenement
id_type_evenement
libelle
description
couleur

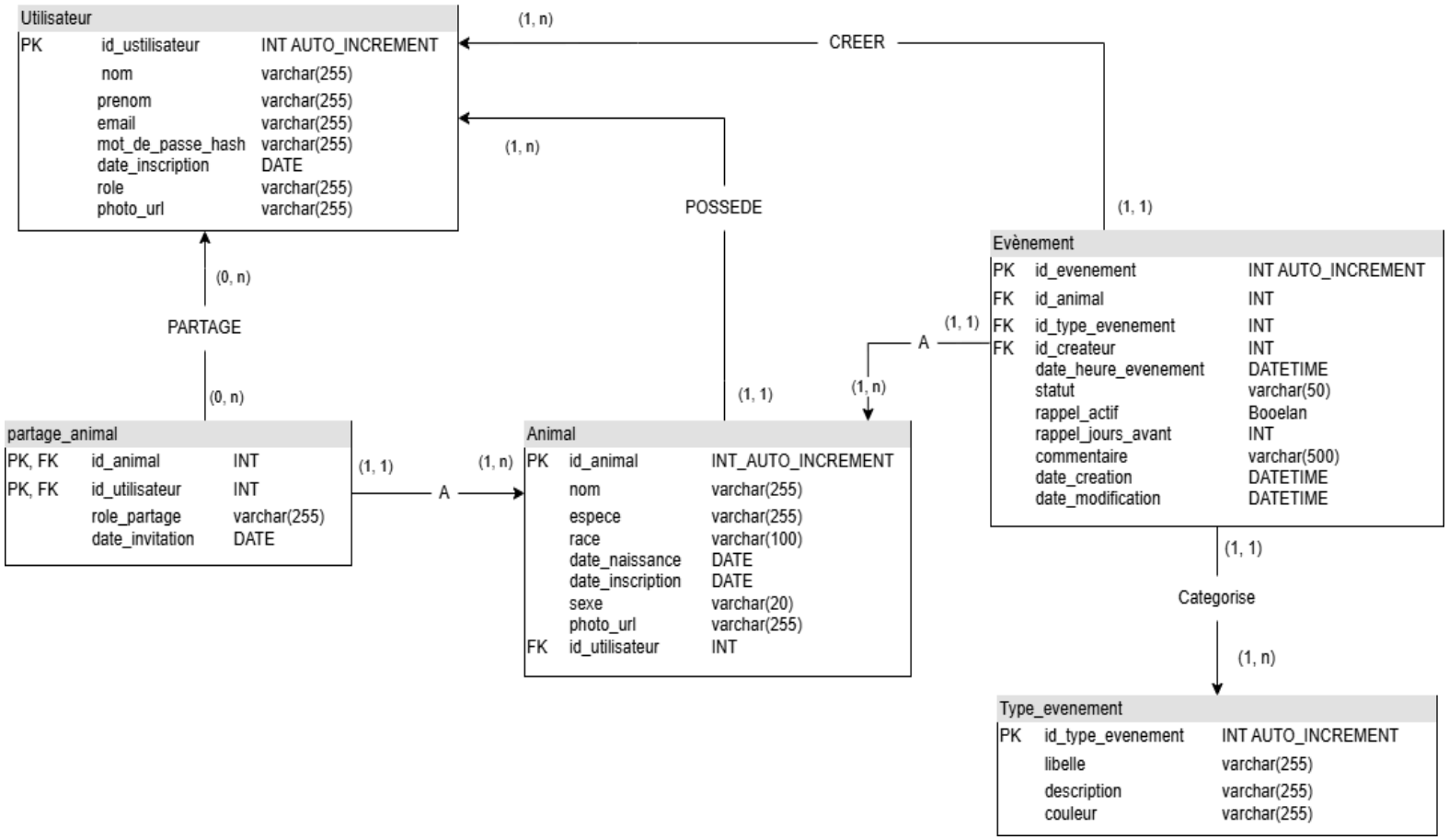
4. Modèle Logique de Données (MLD)

La traduction du MCD en MLD aboutit à la création de cinq tables relationnelles. Les associations 1,N se traduisent par des clés étrangères dans la table du côté N. L'association plusieurs-à-plusieurs (Partage_Animal) génère une table de liaison avec clé primaire composite.

Table	Clé primaire	Clés étrangères	Attributs notables
Utilisateur	id_utilisateur (PK)	—	email (UNIQUE), mot_de_passe_hash, role, photo_url
Animal	id_animal (PK)	id_proprietaire → Utilisateurid_utilisateur → Utilisateur	nom, espece, race, date_naissance, sexe, photo_url
Evenement	id_evenement (PK)	id_animal → Animalid_type_evenement → Type_Evenementid_createur → Utilisateurid_animal → Animal id_type_evenement → Type_Evenementid_type_evenement → Type_Evenement id_createur → Utilisateurid_createur → Utilisateur	date_heure_evenement, statut, rappel_actif, rappel_jours_avant
Type_Evenement	id_type_evenement (PK)	—	libelle, description, couleur
Partage_Animal	id_animal + id_utilisateur (PK composite)	id_animal → Animalid_utilisateur → Utilisateurid_animal → Animal id_utilisateur → Utilisateurid_utilisateur → Utilisateur	role_partage, date_invitation

Vérification des cardinalités : toutes les associations 1,1 se traduisent par une FK NOT NULL dans la table enfant. Les associations 0,N vers des entités optionnelles utilisent des FK NULL lorsque la relation n'est pas obligatoire. La table Partage_Animal garantit l'unicité via sa clé composite (id_animal, id_utilisateur).

MLD


Utilisateur
PK id_ustilisateur INT AUTO_INCREMENT
nom varchar(255)
prenom varchar(255)
email varchar(255)
mot_de_passe_hash varchar(255)
date_inscription DATE
role varchar(255)
photo_url varchar(255)
(1, n)
CREER
(1, n)
POSSEDE
(1, 1)
(0, n)
PARTAGE
(0, n)
Evènement
PK id_evenement INT AUTO_INCREMENT
FK id_animal INT
FK id_type_evenement INT
FK id_createur INT
date_heure_evenement DATETIME
statut varchar(50)
rappel_actif Booelan
rappel_jours_avant INT
commentaire varchar(500)
date_creation DATETIME
date_modification DATETIME
(1, 1)
A
(1, n)
(1, 1)
partage_animal
PK, FK id_animal INT
PK, FK id_utilisateur INT
role_partage varchar(255)
date_invitation DATE
(1, 1)
A
(1, n)
Animal
PK id_animal INT_AUTO_INCREMENT
nom varchar(255)
espece varchar(255)
race varchar(100)
date_naissance DATE
date_inscription DATE
sexe varchar(20)
photo_url varchar(255)
FK id_utilisateur INT
(1, 1)
Categorise
(1, n)
Type_evenement
PK id_type_evenement INT AUTO_INCREMENT
libelle varchar(255)
description varchar(255)
couleur varchar(255)

5. Modèle Physique de Données (MPD)

Le MPD est réalisé pour un SGBD MySQL 8 / MariaDB (moteur InnoDB). Les types SQL sont précisés pour chaque colonne. Les contraintes d'intégrité référentielle sont assurées via des clés étrangères avec ON DELETE CASCADE sur les relations liées à l'animal. Des index sont ajoutés sur les colonnes utilisées fréquemment en jointures et filtres.

```
-- ============================================================
-- TABLE : utilisateur
-- ============================================================
CREATE TABLE utilisateur (
    id_utilisateur    INT           NOT NULL AUTO_INCREMENT,
    nom               VARCHAR(100)  NOT NULL,
    prenom            VARCHAR(100)  NOT NULL,
    email             VARCHAR(255)  NOT NULL,
    mot_de_passe_hash VARCHAR(255)  NOT NULL,
    date_inscription  DATETIME      NOT NULL DEFAULT NOW(),
    role              VARCHAR(50)   NOT NULL DEFAULT 'user',
    photo_url         VARCHAR(500)  NULL,     -- ajout cohérent avec le
dictionnaire
    PRIMARY KEY (id_utilisateur),
    UNIQUE  KEY uk_utilisateur_email (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;


-- ============================================================
-- TABLE : type_evenement
-- ============================================================
CREATE TABLE type_evenement (
    id_type_evenement INT           NOT NULL AUTO_INCREMENT,
    libelle           VARCHAR(100)  NOT NULL,
    description       TEXT          NULL,
    couleur           VARCHAR(7)    NULL,
    PRIMARY KEY (id_type_evenement)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;


-- ============================================================
-- TABLE : animal
-- ============================================================
CREATE TABLE animal (
    id_animal        INT           NOT NULL AUTO_INCREMENT,
    nom              VARCHAR(100)  NOT NULL,
    espece           VARCHAR(100)  NOT NULL,
    race             VARCHAR(100)  NULL,
    date_inscription DATE          NULL,
    date_naissance   DATE          NULL,
    sexe             VARCHAR(10)   NULL,
    photo_url        VARCHAR(500)  NULL,
    id_utilisateur   INT           NOT NULL,
    PRIMARY KEY (id_animal),
    CONSTRAINT fk_animal_utilisateur
        FOREIGN KEY (id_utilisateur) REFERENCES utilisateur(id_utilisateur)
        ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;









-- ============================================================
```

```
-- TABLE : evenement
-- ============================================================
CREATE TABLE evenement (
    id_evenement          INT           NOT NULL AUTO_INCREMENT,
    id_animal             INT           NOT NULL,
    id_type_evenement     INT           NOT NULL,
    id_createur           INT           NOT NULL,
    date_heure_evenement  DATETIME      NOT NULL,
    statut                VARCHAR(50)   NOT NULL DEFAULT 'planifié',
    rappel_actif          BOOLEAN       NOT NULL DEFAULT 0,
    rappel_jours_avant    INT           NULL,
    commentaire           VARCHAR(500)  NULL,
    date_creation         DATETIME      NOT NULL DEFAULT NOW(),
    date_modification     DATETIME      NULL,
    PRIMARY KEY (id_evenement),
    CONSTRAINT fk_evenement_animal
        FOREIGN KEY (id_animal) REFERENCES animal(id_animal)
        ON DELETE CASCADE,
    CONSTRAINT fk_evenement_type
        FOREIGN KEY (id_type_evenement) REFERENCES
type_evenement(id_type_evenement),
    CONSTRAINT fk_evenement_createur
        FOREIGN KEY (id_createur) REFERENCES utilisateur(id_utilisateur),
    -- Index pour optimiser les requêtes de tableau de bord et rappels
    INDEX idx_evenement_animal_date (id_animal, date_heure_evenement),
    INDEX idx_evenement_createur    (id_createur),
    INDEX idx_evenement_rappel      (rappel_actif, date_heure_evenement)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;


-- ============================================================
-- TABLE : partage_animal
-- ============================================================
CREATE TABLE partage_animal (
    id_animal      INT          NOT NULL,
    id_utilisateur INT          NOT NULL,
    role_partage   VARCHAR(50)  NOT NULL,
    date_invitation DATETIME   NOT NULL DEFAULT NOW(),
    PRIMARY KEY (id_animal, id_utilisateur),
    CONSTRAINT fk_partage_animal
        FOREIGN KEY (id_animal) REFERENCES animal(id_animal)
        ON DELETE CASCADE,
    CONSTRAINT fk_partage_utilisateur
        FOREIGN KEY (id_utilisateur) REFERENCES utilisateur(id_utilisateur)
        ON DELETE CASCADE,
    -- Index pour retrouver rapidement tous les animaux partagés avec un
utilisateur
    INDEX idx_partage_utilisateur (id_utilisateur)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

6. Justifications et vérification

Ce modèle répond directement aux besoins fonctionnels du projet. Les justifications suivantes en confirment la cohérence avec le CDCF :

- **Gestion multi-animaux : la relation propriétaire → animaux (1,n) permet à un utilisateur de gérer plusieurs animaux depuis un seul compte.**la relation propriétaire → animaux (1,N) permet à un utilisateur de gérer plusieurs animaux depuis un seul compte.
- **Historique et rappels :** la table Evenement rattachée à un animal conserve l'intégralité de l'historique, permet d'enregistrer des rendez-vous futurs et pilote le système de rappels automatiques via les colonnes rappel_actif et rappel_jours_avant.
- **Normalisation : l'entité Type_Evenement normalise les libellés et évite la duplication des types d'événements. Chaque information est stockée une seule fois à l'endroit le plus pertinent (3NF respectée).**
- **Collaboration multi-utilisateurs :** la table Partage_Animal avec clé composite (id_animal, id_utilisateur) gère les accès différenciés (lecture / modification) conformément à la fonctionnalité F5 du CDCF.
- **Traçabilité :** la colonne id_createur dans Evenement garantit qu'on sait quel utilisateur a créé chaque entrée, ce qui est indispensable dans un contexte multi-utilisateurs.
- **Intégrité référentielle :** ON DELETE CASCADE sur les FK liées à l'animal garantit qu'aucune donnée orpheline ne subsiste lors de la suppression d'un animal.
- **Optimisation des requêtes :** les index composites (id_animal + date_heure_evenement) et ciblés (id_createur, rappel_actif) anticipent les requêtes fréquentes du tableau de bord et du Scheduler CRON de rappels.

La cohérence entre le dictionnaire des données, le MLD et le MPD a été vérifiée : toutes les colonnes du dictionnaire sont présentes dans le script SQL, y compris photo_url dans la table utilisateur. La colonne categorie a été supprimée de Type_Evenement conformément au modèle retenu. Les types de données SQL sont adaptés au SGBD MySQL 8 (InnoDB, utf8mb4).

7. Conclusion

La modélisation de la base de données de l'application PetCare est finalisée. Le respect de la démarche MERISE (MCD → MLD → MPD), la formalisation des règles de gestion métier, la normalisation du schéma et l'attention portée à l'intégrité référentielle et à l'optimisation des index constituent une base technique solide pour le développement backend du Jalon 4.

Les prochaines étapes consisteront à générer les entités Doctrine correspondantes dans Symfony à partir de ce schéma, à configurer les migrations, et à valider la cohérence entre le modèle de données et les diagrammes de classes UML produits au Jalon 4.